

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I ANUSHKA SREE RAJ(192571024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

ANSWERS:

i) Identify the exact condition that causes deadlock.

The deadlock occurs because the two processes acquire the resources in **different orders**:

- **P1** locks **R1** first and then waits for **R2**.
- **P2** locks **R2** first and then waits for **R1**.

If both processes execute simultaneously:

- P1 holds **R1** and waits for **R2**.
- P2 holds **R2** and waits for **R1**.

This creates a **circular wait**, resulting in a deadlock.

ii) Suggest an optimization to eliminate the deadlock without changing process logic. (10)

Use a fixed resource allocation order.

Both processes should acquire resources in the same order:

```
lock(R1)
lock(R2)
Critical Section
unlock(R2)
unlock(R1)
```

Q2. CPU Utilization Problem

A system shows low CPU utilization (30%) even though processes are ready. You analyze the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

ANSWERS:

i) Debug the cause using process management concepts and propose a system-level optimization to improve utilization.

Ready → Waiting → Ready → Waiting

This indicates that processes are **I/O-bound**. They frequently request I/O and enter the waiting state, causing the CPU to remain idle even though processes are active.

Hence, CPU utilization remains low (**30%**).

Increase the **degree of multiprogramming** by allowing more processes into memory.

While one process waits for I/O, another ready process can execute, improving CPU utilization.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

ANSWERS:

i) Identify the memory issue causing performance degradation and propose two optimisations.

Memory Analysis

Memory usage:

- Code = 5 MB
- Heap = 150 MB
- Stack = 1 MB × 25 threads = 25 MB

Total memory used:

$$5 + 150 + 25 = 180 \text{ MB}$$

Although 2 GB RAM is available, frequent page faults indicate poor memory access patterns rather than insufficient RAM.

Possible causes:

- Large heap causing poor locality.
- Multiple threads accessing scattered memory locations.

Q4. It is observed that process P5 never gets CPU time in a multilevel queue

scheduler: • Q1 (Highest Priority): Interactive

- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

i) P5 is in Q3 but keeps getting postponed.

Since higher-priority queues (**Q1** and **Q2**) continuously receive new processes, the scheduler always executes them before Q3.

As a result, **P5 never receives CPU time**, causing **starvation**.

ii) Debug the root cause of starvation and suggest an optimisation to ensure fairness.

To ensure fairness ageing can be implemented

Gradually increase the priority of processes waiting for a long time.

Eventually, P5 is promoted to a higher-priority queue and receives CPU time, ensuring fairness.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead. Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

ANSWER:

i) Explain why context switching is so high and propose two optimisations to reduce overhead. **(10 Marks)**

Reason for High Context Switching

The scheduler uses a 1 ms time quantum, while the average CPU burst is 2 ms.

As a result:

- Threads are preempted before completing execution.
- Very frequent I/O operations also trigger additional context switches.

Therefore, the system performs 20,000 context switches per second, creating high scheduling overhead.

Optimizations methods are:

1. Increase the Time Quantum

Increase the quantum from **1 ms** to **4–5 ms**.

This reduces unnecessary preemption and context-switch overhead.

2. Use a Multilevel Feedback Queue (MLFQ)

Assign longer time quanta to CPU-bound threads and shorter quanta to interactive or I/O-bound threads.

This reduces context switches while maintaining good responsiveness.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues

Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (100%)
Q1						
Q2						
Q3						
Q4						
Q5						

Total						
--------------	--	--	--	--	--	--